

AI Engineering Assignment

RFP Intelligence Platform: RAG Search Engine & Multi-Agent System

1. Objective

Participants will design and build an **RFP Intelligence Platform** that understands bid / Request for Proposal (RFP) documents and makes them searchable, answerable and extractable using Large Language Models.

The assignment has **two main goals**:

- **Build a Search Engine for the RAG system** – a retrieval layer that indexes all bid documents (HTML and PDF, including addendums) and returns the most relevant, cited passages for any query. This search engine is the foundation that every LLM answer must be grounded in.
- **Build a Multi-Agent System** – a set of cooperating, specialized AI agents, coordinated by an orchestrator, that use the search engine as a tool to extract structured bid information, reconcile addendums, validate results and answer user questions.

The end result must be able to (a) produce a structured JSON record for each bid, with every value backed by a source citation, and (b) answer free-form natural-language questions about the bids.

2. Background

Sales and bid teams receive many RFPs in different formats. Key facts (due dates, bond requirements, product specifications, required affidavits) are scattered across the main RFP, addendums and portal web pages, and they often change between versions. Reading them manually is slow and error-prone. A good system must find the right information quickly, understand when an addendum overrides the original document, and never invent facts that are not in the documents.

3. Documents Provided

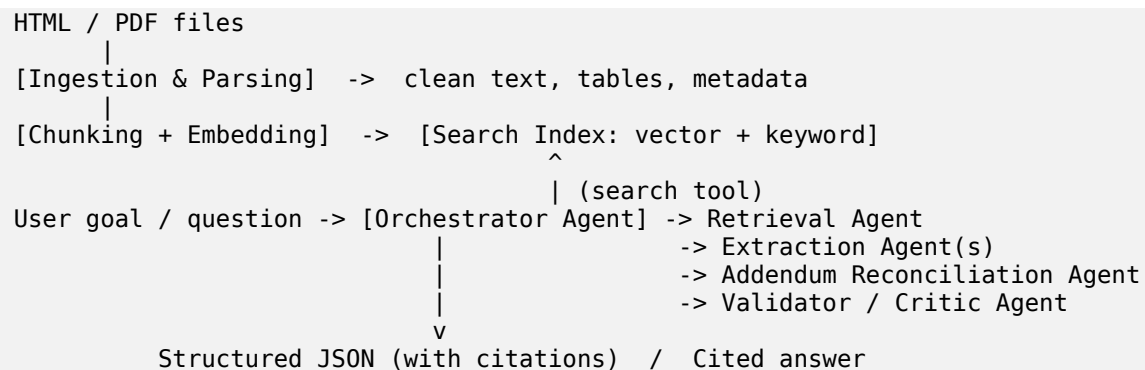
Two bid folders are provided. Each folder represents one bid and must be processed as one unit.

Folder	Files
Bid1	Student and Staff Computing Devices – BidNet Direct bid page (HTML); JA-207652 Student and Staff Computing Devices FINAL (PDF); Addendum 1 (PDF); Addendum 2 (PDF)
Bid2	Dell Laptops w/ Extended Warranty – BidNet Direct bid page (HTML); PORFP – Dell Laptop Final (PDF); Dell Laptop Specs (PDF); Contract Affidavit (PDF); Mercury Affidavit (PDF)

Note: The solution must not be hard-coded to these two bids. It will also be tested on at least one **unseen bid folder** with a similar structure.

4. High-Level Architecture

Your solution is expected to follow (or improve on) the flow below:



5. Part A – Document Ingestion & Parsing

- Parse **HTML** bid pages (e.g. BeautifulSoup, trafilatura) and **PDF** files (e.g. PyMuPDF, pdfplumber, Unstructured, Docling). Keep the page structure where possible.
- Extract **tables** correctly (specification tables and bid information tables are important).
- Clean the text: remove repeated headers/footers, fix broken lines and hyphenation, normalize whitespace.
- Attach **metadata** to every document: bid_id, file_name, doc_type (bid_page, rfp, addendum, specs, affidavit), addendum_number, page_number, and document date where available.
- Handle failure cases gracefully (scanned pages, empty pages, unreadable files) and log them. OCR support is a bonus.

6. Part B – RAG Search Engine (Main Goal)

Build a real search engine over the bid documents, not only a single vector lookup. It must be usable on its own (as an API or CLI) and as a **tool** that the agents call.

6.1 Indexing

- Implement a **chunking strategy** suited to RFPs (e.g. section/heading-aware or table-aware chunking with overlap). Justify your chunk size and strategy in the README.
- Generate embeddings with an embedding model of your choice (open-source or API-based).
- Store chunks in a vector store (e.g. FAISS, Chroma, Qdrant, Milvus, pgvector) together with their metadata.
- Support **incremental indexing**: adding a new bid folder must not require re-indexing everything.

6.2 Retrieval

- **Hybrid search**: combine semantic (dense vector) search with keyword search (BM25 or similar). Keyword search is important for exact values like bid numbers, part numbers and model numbers. Merge results (e.g. Reciprocal Rank Fusion or weighted scores).

- **Metadata filtering:** restrict searches by bid, document type, addendum number, etc. (e.g. "only search Bid2 specs").
- **Re-ranking:** re-rank the top candidates with a cross-encoder or LLM-based re-ranker.
- **Query understanding:** query rewriting / expansion (e.g. "deadline" → "due date, submission deadline, closing date").
- **Citations:** every result must return the source file, page number and chunk text so answers can be traced back.

6.3 Search Interface

- Expose the search engine through a **REST API** (e.g. FastAPI) and/or CLI. Example endpoints: POST /index, GET /search?q=...&bid_id=...&top_k=5, POST /ask.
- A simple web UI (e.g. Streamlit, Gradio) to search and chat with the bids is a bonus.

6.4 Retrieval Evaluation

- Create a small **evaluation set** of at least **15 question → expected source passage** pairs covering both bids.
- Report **Recall@k**, **MRR** (or nDCG) and compare at least two configurations (e.g. vector-only vs. hybrid vs. hybrid + re-ranker). Include the results table in the README.

7. Part C – Multi-Agent System (Main Goal)

Design a multi-agent system where each agent has a clear, single responsibility, and the agents collaborate to complete the task. Agents must use the RAG search engine as a tool instead of receiving whole documents in the prompt.

7.1 Required Agents

The following roles are expected. You may merge, split or add agents if you explain why.

Agent	Responsibility	Output
Orchestrator / Planner	Receives the user goal ("extract bid" or a question), breaks it into sub-tasks, routes them to the right agents, and decides when the job is done.	Execution plan, final response
Ingestion Agent	Detects file type, parses HTML/PDF (including tables), cleans text, attaches metadata and triggers indexing.	Normalized documents + chunks
Retrieval / Search Agent	Rewrites and expands queries, calls the search engine, applies metadata filters and reranking, returns evidence with citations.	Ranked evidence passages
Extraction Agent(s)	Fills the structured fields using only retrieved evidence. May be split into specialists (e.g. Dates & Logistics, Commercial & Legal, Product & Specs).	Field values + source citations
Addendum Reconciliation Agent	Compares the base RFP with its addendums and applies the latest valid change to each field (e.g.	Change log + updated fields

Agent	Responsibility	Output
	extended due date).	
Validator / Critic Agent	Checks each field for evidence support, format (dates, numbers), consistency and hallucination. Sends failed fields back for re-extraction.	Validation report, confidence scores
Q&A / Report Agent	Answers free-form user questions over the bids and produces the final JSON and a human-readable summary.	Answer with citations, JSON output

7.2 Orchestration Requirements

- Use any agent framework (e.g. **LangGraph**, **CrewAI**, **AutoGen**, **OpenAI Agents SDK**, **Claude Agent SDK**, **LlamaIndex Agents**) or build your own orchestration. Explain the choice.
- Define a **shared state / memory** object that holds the plan, retrieved evidence, draft fields, validation results and final output.
- Implement a **feedback loop**: when the Validator rejects a field, the Orchestrator must re-run retrieval/extraction for that field (with a maximum number of retries to avoid infinite loops).
- Agents should run **in parallel** where possible (e.g. field groups extracted concurrently).
- Use **structured outputs** (e.g. Pydantic models / JSON schema) for all agent-to-agent messages.
- **Guardrails**: agents must answer "Not found in documents" instead of guessing; every value must have at least one citation.
- **Observability**: log each agent step (input, tool calls, output, tokens, latency). Tools like LangSmith, Langfuse or simple structured logs are acceptable. Include one example trace in the submission.
- Handle errors (LLM timeout, bad JSON, empty retrieval) without crashing the whole pipeline.

7.3 Supported Tasks

The multi-agent system must support at least these two modes:

1. **Extraction mode** – given a bid folder, produce the complete structured JSON record (Section 8).
2. **Question-answering mode** – answer natural-language questions with citations, for example:
 - "What is the submission deadline for Bid1 after all addendums?"
 - "Which affidavits are required for the Dell laptop bid?"
 - "Compare the warranty requirements of both bids."
 - "Is a bid bond required, and if so, how much?"
 - "What changed in Addendum 2 compared to the original RFP?"

8. Part D – Structured Information Extraction

For every bid, the system must extract the following fields. The **Description** column explains what is expected for each field.

Field	Description
Bid Number	Official solicitation / RFP identifier (e.g. JA-207652).
Title	Full title of the bid or RFP.
Due Date	Final submission deadline, including time and time zone. Must reflect any addendum changes.
Bid Submission Type	How the bid is submitted (electronic portal, sealed paper, email, etc.).
Term of Bid	Contract duration and any renewal options.
Pre Bid Meeting	Date, time, location and whether attendance is mandatory, or "None".
Installation	Whether installation / deployment / imaging services are required.
Bid Bond Requirement	Required bid bond or security, amount or percentage, or "Not required".
Delivery Date	Required delivery date or delivery window after award.
Payment Terms	Payment terms (e.g. Net 30), invoicing conditions.
Any Additional Documentation Required	Affidavits, forms, certificates, insurance, W-9, etc.
MFG for Registration	Manufacturer(s) the bidder must be registered / authorized with.
Contract or Cooperative to use	Referenced cooperative / state contract vehicle, if any.
Model_no	Model number(s) of the requested product(s).
Part_no	Part / SKU number(s) of the requested product(s).
Product	Product name(s) and quantity.
contact_info	Procurement contact: name, email, phone.
company_name	Issuing organization / agency name.
Bid Summary	A concise (3-6 sentence) summary of the bid, generated by the system.
Product Specification	Key technical specifications (CPU, RAM, storage, display, warranty, etc.).

8.1 Output Format

Output one JSON file per bid. Each field must contain the value, its source(s) and a confidence score. Use null with a reason when a field is not present in the documents. Example:

```
{
  "bid_id": "Bid1",
  "fields": {
    "Due Date": {
      "value": "YYYY-MM-DD HH:MM TZ",
```

```

    "sources": [{"file": "Addendum 2 ....pdf", "page": 1}],
    "confidence": 0.93,
    "notes": "Extended by Addendum 2 (original date: YYYY-MM-DD)"
  },
  "Bid Bond Requirement": {
    "value": null,
    "sources": [],
    "confidence": 0.80,
    "notes": "Not found in documents"
  }
},
"addendum_changes": [ ... ],
"validation": { "passed": 18, "failed": 0, "not_found": 2 }
}

```

(Values above are illustrative only.)

9. Technical Requirements

- Python 3.10+. Any LLM provider (OpenAI, Anthropic, Gemini, open-source via Ollama / vLLM, etc.) may be used.
- All API keys and model names must come from environment variables or a config file – **no secrets in the code**.
- The system must be runnable with **one command** after setup (e.g. `python main.py --bid ./Bid1` or `docker compose up`).
- Modular project structure, e.g. `ingestion/`, `search/`, `agents/`, `api/`, `eval/`, `tests/`.
- Unit tests for the parsing and search components at minimum.
- Must work on a new, unseen bid folder without code changes.

10. Evaluation Criteria

Criterion	What we look for	Weight (%)
RAG Search Engine	Retrieval quality (Recall@k / MRR on your test set), hybrid search, reranking, metadata filtering, citations, search API/UI.	25
Multi-Agent System	Clear agent roles, sound orchestration, shared state, validation/feedback loop, error handling, observability.	25
Extraction Accuracy	Correctness and completeness of the 20 fields for both bids, including addendum updates.	20
Robustness	Handles different document structures, missing fields, tables, long documents and new unseen bids without code changes.	10
Code Quality & Architecture	Modular, typed, tested, documented code; configuration instead of hard-coding; reproducible setup.	10
Documentation & Demo	README, architecture diagram, design decisions, evaluation results and demo.	10

11. Deliverables

- **Source code** (Git repository) containing the ingestion pipeline, the RAG search engine, the multi-agent system and the API/CLI.
- **README** with setup instructions, dependencies, how to run each mode, and design decisions (chunking, embedding model, retrieval strategy, agent framework, prompts).
- **Architecture diagram** showing the search engine, the agents and how they communicate.
- **JSON output files** – one per bid folder – with values, citations and confidence scores.
- **Retrieval evaluation report** – the question set and Recall@k / MRR results for each configuration tested.
- **Sample Q&A log** – at least 10 questions with the system's cited answers.
- **Agent trace** – one full example trace of an extraction run.
- **Demo video** (5–10 minutes) walking through the system, or a live demo session.

12. Bonus (Optional)

- Web UI for search and chat (Streamlit / Gradio / React).
- OCR for scanned PDFs and image-based tables.
- Bid comparison agent that produces a side-by-side comparison report of multiple bids.
- Automatic "go / no-go" recommendation based on configurable company capabilities.
- Semantic caching of repeated queries and cost/latency tracking per run.
- Deployment with Docker and a CI pipeline that runs the tests and retrieval evaluation.

13. Submission Guidelines

- Share a Git repository link (public or with access granted) containing all deliverables.
- Clearly list any assumptions you made and any known limitations.
- Use of AI coding assistants is allowed, but you must understand and be able to explain every part of your solution during the review.
- The submission deadline will be communicated separately.